

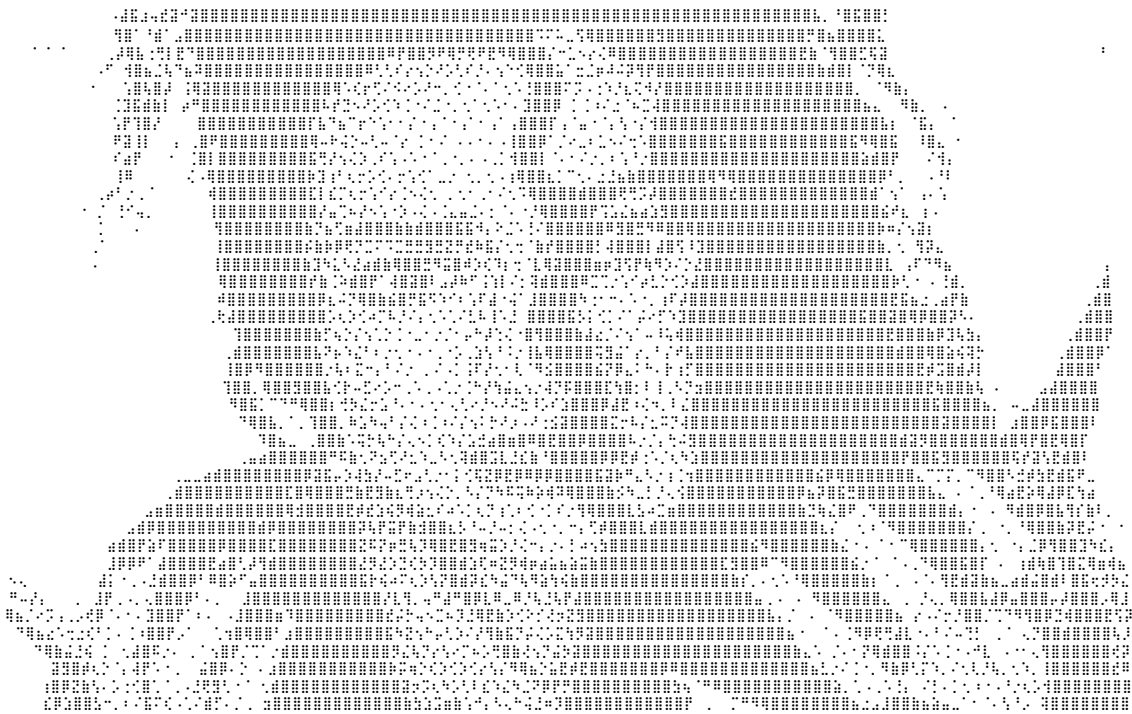
INSTITUTO FEDERAL DA PARAÍBA
CAMPUS CAMPINA GRANDE
BACHARELADO EM ENGENHARIA DA COMPUTAÇÃO
DISCIPLINA DE ESTRUTURAS DE DADOS
PROF. VICTOR ANDRÉ PINHO DE OLIVEIRA

Estruturas de Dados

Atividade Prática EXTRA - Listas Lineares Sequenciais

Instruções

Responda às questões abaixo. Pode usar este próprio documento. Questões práticas devem ser anexadas separadamente.



Legenda:

Quirrell: Questões mais simples e diretas

Olho-Tonto: Questões de nível intermediário

Dolores: Questões de nível mais difícil, exigentes

Voldemort: Questões com nível de exigência altíssimo, desafiadoras

Questões

Para as questões a seguir, use como base a sua solução da questão Voldemort da Atividade 4, mas admita, para os propósitos das questões que seguem, que a lista possa inserir elementos de forma não Ordenada.

1. **Quirrell** Adicione uma função chamada *contar* que recebe um valor inteiro e devolve a quantidade de ocorrências deste valor na Lista.

```
int contar(LISTA *lst, int valor);
```

<https://www.programiz.com/online-compiler/4udPGck8PHEVg>

2. **Quirrell** Adicione uma função chamada *pop* que remove e retorna o último elemento da Lista. A função só funciona se houver pelo menos um elemento na Lista.

```
int pop(LISTA *lst);
```

<https://www.programiz.com/online-compiler/6Rcs6tm1ICVL8>

3. **Olho-Tonto** Adicione uma função chamada *inserirPos* que recebe uma posição *pos* e um *valor* inteiro a ser inserido na Lista. A função só deve inserir *valor* se *pos* for uma posição válida na Lista.

```
void inserirPos(LISTA *lst, int pos, int valor);
```

<https://www.programiz.com/online-compiler/39JnCHTdni2xH>

4. **Dolores** Adicione uma função chamada *copiar* que recebe duas Listas e faz uma cópia da segunda na primeira. Cuidado com os ponteiros e a alocação de memória!

```
void copiar(LISTA *lst1, LISTA *lst2);
```

<https://www.programiz.com/online-compiler/9QRdHXa5njkQF>

5. **Dolores** Adicione uma função chamada *estender* que recebe duas Listas e estende, isto é, anexa uma cópia dos elementos da segunda a partir do fim da primeira. Avalia se a primeira Lista suporta anexar os elementos da segunda.

```
void estender(LISTA *lst1, LISTA *lst2);
```

<https://www.programiz.com/online-compiler/9QRdHXa5njkQF>

6. **Dolores** Adicione uma função chamada *inverter* que recebe duas Listas, e armazena na primeira os elementos da segunda na sua ordem inversa. Obs.: se a primeira Lista tiver elementos, estes serão perdidos, mas cuidado com a alocação dinâmica. Avaliar também se a primeira Lista suporta receber a segunda Lista.

```
void inverter(LISTA *lst1, LISTA *lst2);
```

<https://www.programiz.com/online-compiler/0qxRv9ZXX5ooQ>

7. **Dolores** Adicione uma função utilitária chamada *redimensionar* que recebe um novo valor *MAX* para o array interno. Esta função deverá redimensionar a memória alocada dinamicamente, para mais ou para menos. Contudo, não deve ser permitido reduzir o tamanho do array se houver a possibilidade de perda de elementos da Lista.

```
void redimensionar(LISTA *lst1, int MAX);
```

<https://www.programiz.com/online-compiler/5Z29brEvGcZks>

8. **Dolores** Reimplemente as funções (adicione um v2 ao final do nome da função) **inserir**, **inserirPos**, **copiar**, **estender** e **inverter** de modo a fazer uso da função utilitária **redimensionar**, garantindo a operação/funcionalidade das funções. Ou seja, o tamanho do array deixará de ser um problema e a lista passará a caber uma quantidade indefinida de elementos, a menos que não seja possível, de fato, alocar mais memória.

<https://www.programiz.com/online-compiler/9vpq62iUeawfs>

9. **Voldemort** Adicione uma função chamada *ordenar* que recebe uma Lista e ordena seus elementos em ordem crescente.

```
void ordenar(LISTA *lst);
```

<https://www.programiz.com/online-compiler/9vpq62iUeawfs>